DECK 3 OF 10 · FRC JAVA CRASH COURSE

# AdvantageScope: see what your robot is doing

Reading robot data live and from logs

WPILib 2026 · Java · Command-based

OVERVIEW

# What this deck covers

### Connect

Open AdvantageScope and connect it to a robot.

### Publish

Send values out of your Java code so you can graph them.

### Save

Record a log, get it off the robot, and open it.

### Read

Read a graph to see what the robot did, and how well.

WHY A DATA VIEWER

# Print statements versus a data viewer

| Print statements | A data viewer |
| --- | --- |
| The robot loop runs 50 times a second, so lines fly past. | Every value becomes a line you can read at a glance. |
| You see one number at a time, with no shape to it. | Everything shares one clock, so causes line up with effects. |
| Two values are hard to match up at the same moment. | You can pause, zoom in, and look again after the match. |

GETTING CONNECTED

# Starting AdvantageScope and connecting to a robot

1. **Open it**
   In WPILib VS Code: Ctrl+Shift+P, then WPILib: Start Tool, then AdvantageScope.

2. **Set your robot address once**
   App > Show Preferences... > Robot Address. Team 1212 is 10.12.12.2.

3. **Connect**
   File > Connect to Robot > Default: NetworkTables 4, or press Ctrl+K.

4. **Watch the window title**
   It says Searching until the robot answers.

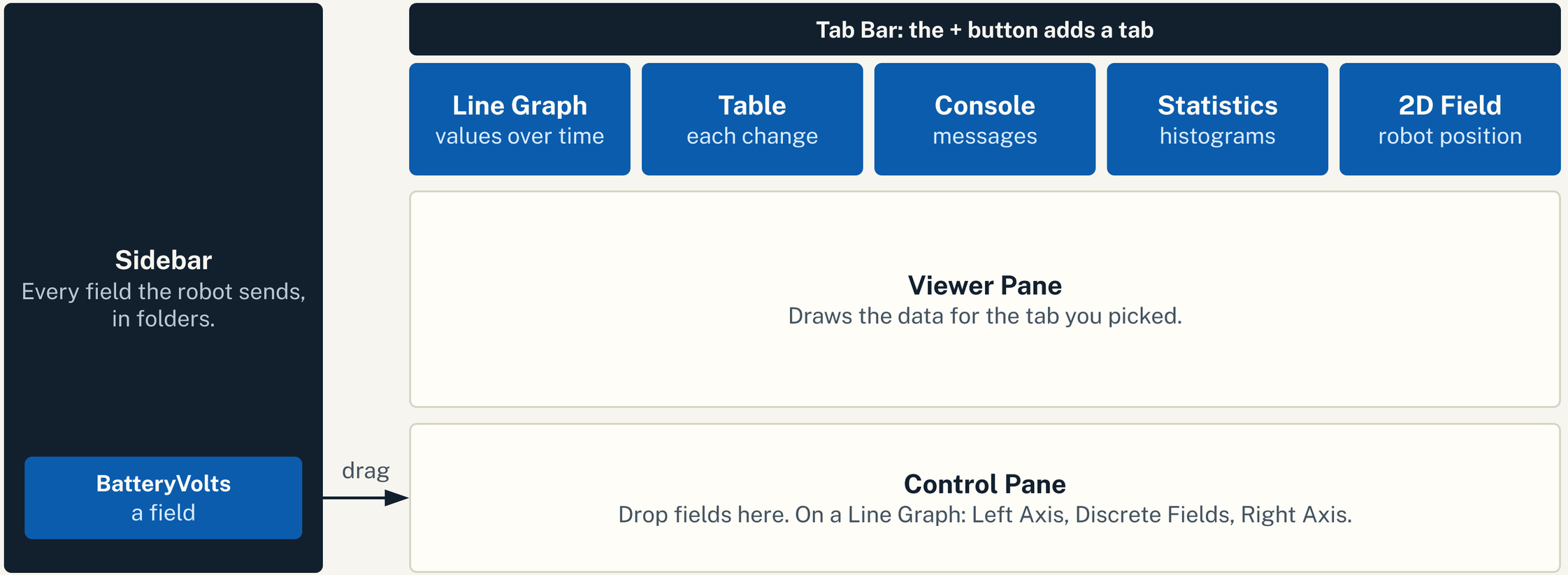
5. **On a USB cable?**
   File > Use USB roboRIO Address switches to 172.22.11.2.

**Live Sources docs**

docs.advantagescope.org

THE INTERFACE

# The AdvantageScope window




Schematic, not a screenshot. Part and tab names come from the AdvantageScope docs and source.

LEVEL 1: SEE IT LIVE

# Publishing a value with SmartDashboard

CANDriveSubsystem.java

```java
@Override
public void periodic() {
  // Runs every 20 ms, even while the robot is disabled.
  SmartDashboard.putNumber("Drive/LeftAppliedOutput", leftLeader.getAppliedOutput());
  SmartDashboard.putNumber("Drive/RightAppliedOutput", rightLeader.getAppliedOutput());
  SmartDashboard.putNumber("Drive/BatteryVolts", RobotController.getBatteryVoltage());
}
```

The key becomes a field. In the sidebar, open SmartDashboard, then Drive.



**2027: SmartDashboard is removed. A Telemetry API replaces it.**

LEVEL 2: SAVE IT TO A FILE

# Saving a log with DataLogManager

Robot.java

```java
public Robot() {
  // Save every NetworkTables change to a .wpilog file.
  DataLogManager.start();

  // Also record the enabled state and the joysticks.
  DriverStation.startDataLog(DataLogManager.getLog());

  // Level 3, next slide: log every @Logged class.
  Epilogue.bind(this);
}
```

By default the robot saves nothing. These two lines change that. The last line is next.

LEVEL 3: THE 2026 BEST PRACTICE

# Logging whole classes with Epilogue

CANDriveSubsystem.java

```java
1 @Logged
  public class CANDriveSubsystem extends SubsystemBase {
2   // Robot() calls Epilogue.bind(this), which reads these.
3   public double getLeftCurrentAmps() {
      return leftLeader.getOutputCurrent();
    }

3   public double getRightCurrentAmps() {
      return rightLeader.getOutputCurrent();
    }

4   @Logged(name = "BrownedOut")
    public boolean isBrownedOut() {
      return RobotController.isBrownedOut();
    }
```

1. **Tag the class**
   Tag Robot and RobotContainer too.

2. **One line starts it**
   Epilogue.bind(this), from the last slide.

3. **Public, no arguments**
   These methods are logged for you, and so are fields.

4. **Pick a name**
   name sets what you will see in the log.

LOGS

# Getting a log off the robot

1. **Best: a USB stick in the roboRIO**
   FAT32, in place before code starts. Logs go in a logs folder on the stick.

2. **No stick?**
   Logs go to /home/lvuser/logs. WPILib warns against this on a roboRIO 1.

3. **Copy it**
   Pull the stick with the robot off, or use File > Download Logs....

4. **Check the folder**
   Robot Log Path starts as /U/logs, the stick. Change it for internal storage.

5. **Open it**
   File > Open Log(s)... REV .revlog and CTRE .hoot files open the same way.



**Log Files docs**

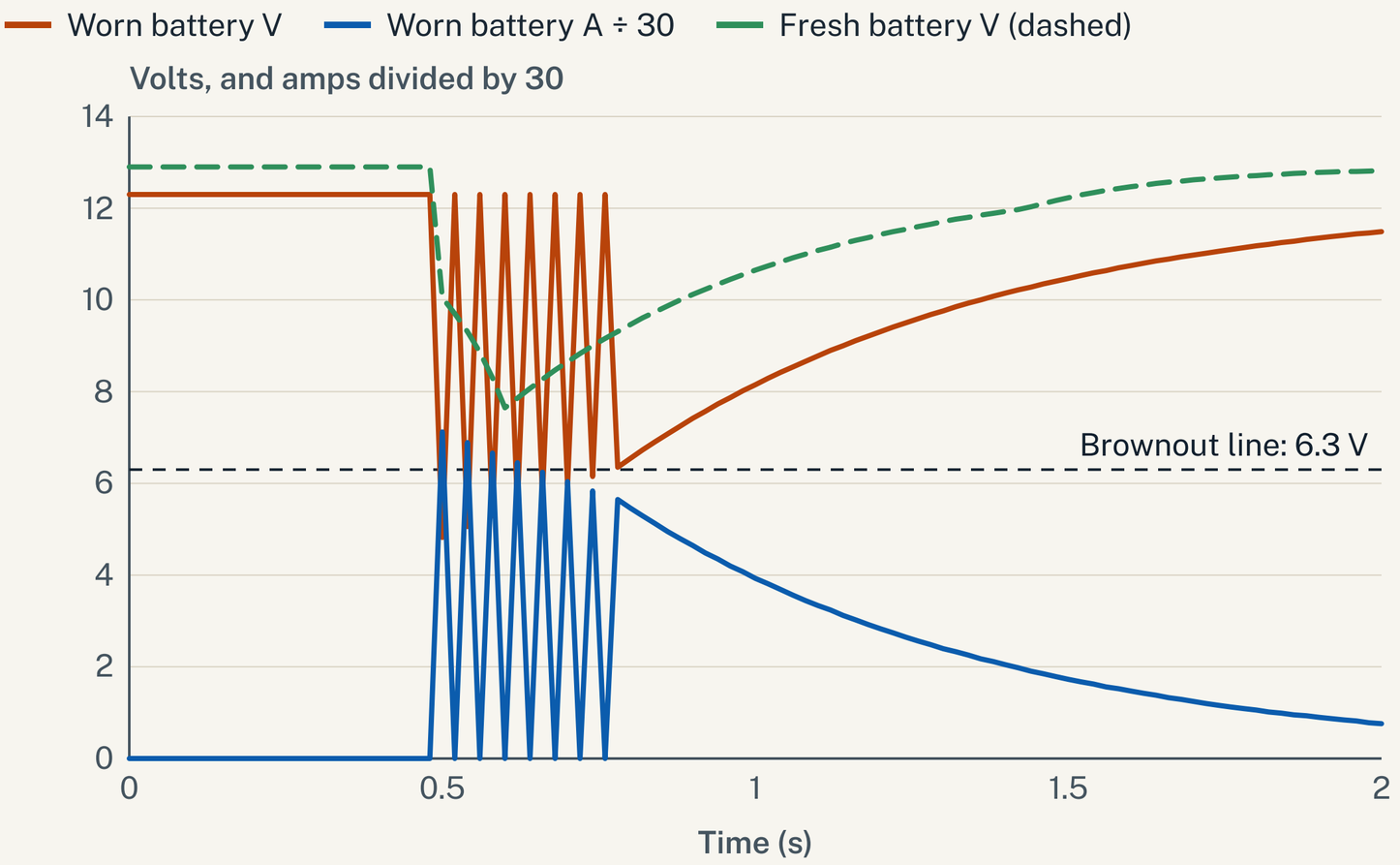
docs.advantagescope.org



**2027: Systemcore alphas log to /home/systemcore/logs without a stick.**

WORKED EXAMPLE, NOT A REAL EVENT

# Reading a brownout from voltage and current




1. **The problem**
   The robot stutters on a hard launch.

2. **Voltage**
   Falls to 4.8 V, under the line.

3. **Current**
   Jumps to about 210 A at the same moment.

4. **The fix**
   A fresh battery stays above 7.6 V here.

BEST PRACTICE

# What to log and how to name it

> **Log inputs and outputs, put units in the names, and log more than you think you need.**

You cannot record something after the match is over. Inputs are what the robot sensed. Outputs are what it asked for. With both, you can tell whether the code or the machine went wrong.

```
Drive/Left              // Left what? How big?
Drive/LeftCurrentAmps   // Clear, and AdvantageScope labels the axis
```

COMMON MISTAKE

# When nothing shows up live

| What you see | Why it happens | The fix |
| --- | --- | --- |
| Title stuck on Searching | Wrong Robot Address. A new install starts at 10.00.00.2. | Set 10.TE.AM.2 with your own team number. |
| Searching, on a USB cable | The USB address is not the radio address. | File > Use USB roboRIO Address. |
| Searching, or an empty sidebar | Wrong mode: Connect to Simulator, or the AdvantageKit, Phoenix or RLOG source. | File > Connect to Robot > Default: NetworkTables 4. |
| Connected, but your field is missing | The code never published it, or was never deployed. | Add the put to periodic() and deploy again. |

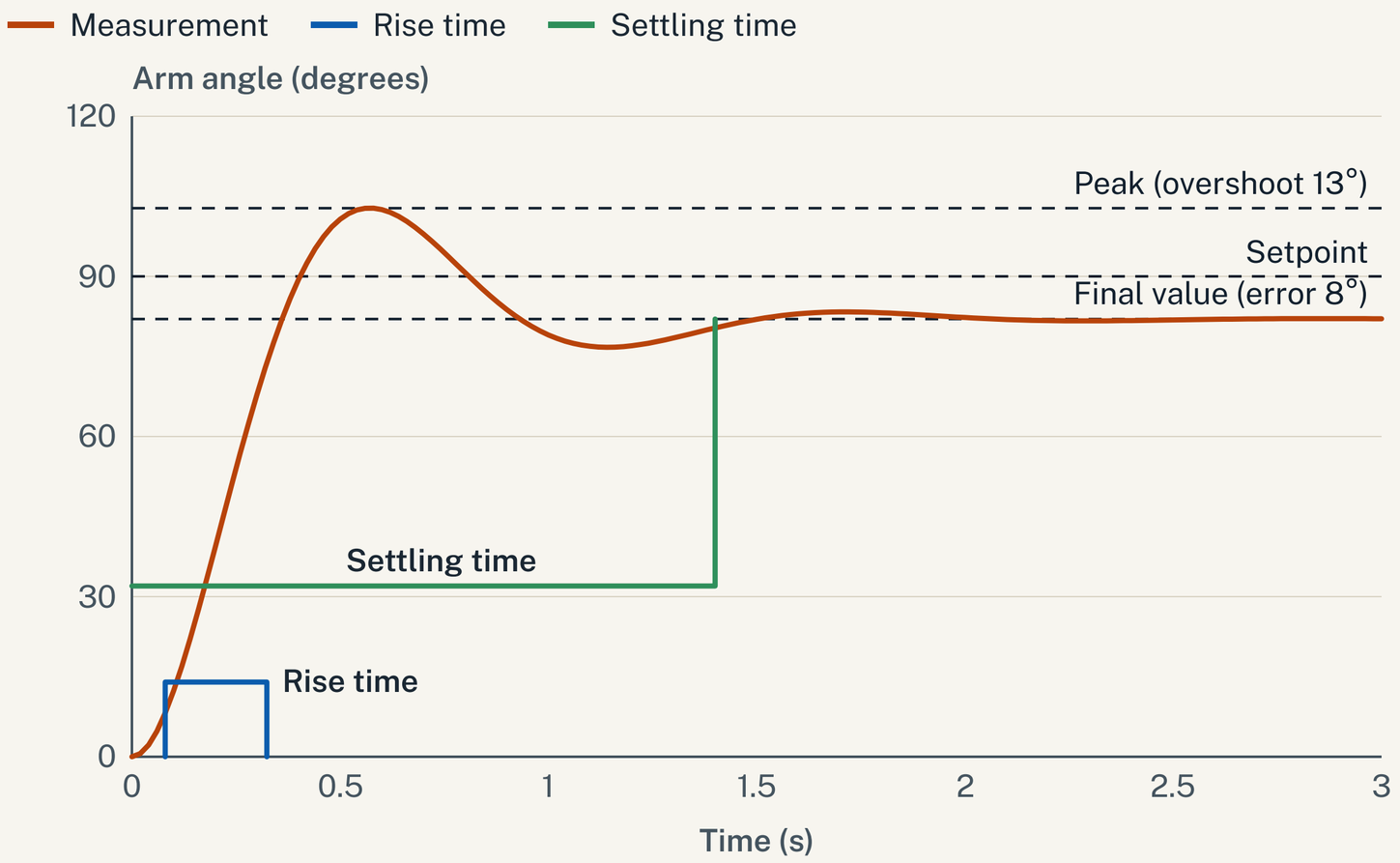
COMMON MISTAKE

# The log is empty or missing

| | |
|---|---|
| **What you see** | Failed to open log folder or No files found in folder, or a file with almost nothing in it. |
| **Why it happens** | Logging never started, no Driver Station was connected, or Robot Log Path is the wrong folder. |
| **The fix** | Call DataLogManager.start(), connect the Driver Station, and match the path to where logs really are. |

CONTROL THEORY

# Reading a response on a line graph




1. **Rise time**
   0.24 s to go from 10% to 90%.

2. **Overshoot**
   Swings 13 degrees past the setpoint.

3. **Settling time**
   1.4 s until it stays within 2%.

4. **Steady-state error**
   Stops 8 degrees short.

EXERCISE

# Graphing left and right applied output

1. Copy periodic() from Level 1 into your CANDriveSubsystem, add the imports, and deploy.
2. Put the robot on blocks, wheels off the floor. **No robot?** getAppliedOutput() reads 0 in the simulator, so run Level 3 and graph Left Motor Speed under Robot/container/drive/drive.
3. Connect AdvantageScope, then open SmartDashboard and Drive in the sidebar.
4. Drag LeftAppliedOutput and RightAppliedOutput to Left Axis.
5. Enable. Push the left stick forward, then use the right stick to turn.

**Done when: both lines follow the sticks, and they move differently when you turn.**

NEXT SEASON

# What changes for 2027

| In 2026 | Changing for 2027 |
| --- | --- |
| roboRIO, logs in /home/lvuser/logs | Systemcore, logs in /home/systemcore/logs |
| SmartDashboard.putNumber | SmartDashboard removed. A Telemetry API replaces it. |
| Packages under edu.wpi.first | Packages under org.wpilib, including Epilogue |
| AdvantageScope for debugging | AdvantageScope and Elastic are the recommended dashboards |

WPILib 2027 is in alpha, so any of this may change. Everything in this deck is right for 2026.

RECAP

# What this deck covered

- Connecting AdvantageScope, and the parts of its window.
- Publishing with SmartDashboard, saving with DataLogManager, and tagging classes with @Logged.
- Getting a .wpilog off the robot and opening it.
- Reading a brownout from voltage and current.
- Rise time, overshoot, settling time and steady-state error on a graph.

**Next: Deck 4 drives the simulator in AdvantageScope, with no robot needed.**

REFERENCE

# Official docs



**AdvantageScope docs**

docs.advantagescope.org



**On-robot data logs**

docs.wpilib.org



**Epilogue**

docs.wpilib.org



**Brownouts**

docs.wpilib.org